

Implementação do Protocolo “PowerUDP”

Introdução às Redes de Comunicação · DEI · Universidade de Coimbra · 2025/2026

Francisco Martins Bernardo
2024210034

José Pedro Clemêncio
2024220132

1. Introdução e Arquitetura

O presente trabalho implementa o protocolo **PowerUDP** — uma extensão ao UDP que acrescenta garantias de fiabilidade: retransmissão automática com *exponential backoff*, ordenação de mensagens por número de sequência e reconfiguração dinâmica da rede via multicast. O sistema foi desenvolvido em C com sockets, no cenário de rede da Meta 1 (três routers Cisco R1/R2/R3, dois switches, quatro containers Docker Linux).

A solução é composta por três componentes principais:

Componente	Protocolo	Função
Cliente	PowerUDP (UDP) + TCP	Envia/recebe mensagens fiáveis; regista-se no servidor via TCP
Servidor	TCP + Multicast UDP	Autentica clientes via PSK; difunde nova configuração por multicast
PowerUDP	UDP + cabeçalho próprio	Retransmissão, backoff exponencial, sequenciação e ACK/NAK

2. Protocolo PowerUDP

2.1 Cabeçalho e Encapsulamento

Para implementar os mecanismos de fiabilidade, foi adicionado um cabeçalho próprio a cada mensagem antes de ser inserida no payload UDP, conforme a Figura 3 do enunciado. O cabeçalho foi definido com três campos:

```
[ type (1B) | seq_num (4B) | data_len (2B) ] + dados da aplicação → payload UDP
```

type distingue DATA, ACK e NAK;

seq_num identifica cada mensagem de forma única;

data_len indica o comprimento dos dados que se seguem. Todos os campos multi-byte são convertidos para network byte order com *htonl()/htons()* antes do envio e convertidos no sentido inverso na receção.

2.2 Modelo Stop-and-Wait

O PowerUDP utiliza um modelo *stop-and-wait*: o emissor só transmite a mensagem seguinte após receber confirmação (ACK) da anterior. Esta opção garante a ordenação e simplifica a gestão do número de sequência, evitando ambiguidades. O recetor responde com **ACK** se o número de sequência for o esperado, ou com **NAK** se for duplicado ou fora de ordem. A espera pelo ACK é implementada com *select()* com timeout configurável, evitando bloqueio indefinido e permitindo a deteção de perda de pacotes.

2.3 Retransmissão e Exponential Backoff

Quando o ACK não chega dentro do timeout, o pacote é retransmitido até *max_retries* vezes. O tempo de espera entre tentativas segue a fórmula do enunciado:

$$T_n = T_{min} \times 2^n \quad (n = \text{número de tentativas falhadas})$$

Com os valores por omissão de **Tmin = 500 ms** e **max_retries = 4**, os timeouts sucessivos são: 500 ms → 1000 ms → 2000 ms → 4000 ms → 8000 ms. O backoff pode ser desativado via *ConfigMessage*, caso em que o timeout permanece fixo em Tmin. Tanto a retransmissão como o backoff são funcionalidades independentes e configuráveis.

2.4 Ordenação e NAK

Cada mensagem enviada incrementa o número de sequência do emissor. O recetor mantém o valor esperado e rejeita qualquer mensagem cujo *seq_num* seja diferente, respondendo com NAK. Esta abordagem garante a entrega ordenada e evita que duplicados (resultantes de retransmissões) sejam processados pela aplicação. O emissor, ao receber um NAK, retransmite imediatamente sem incrementar o contador de tentativas falhadas.

3. Servidor e Configuração via Multicast

3.1 Servidor TCP com fork()

O servidor segue o modelo clássico dos slides TP05: *socket()* → *bind()* → *listen()* → *accept()* → *fork()*. Cada cliente é tratado num processo filho independente, o que permite múltiplas ligações simultâneas sem bloqueio. Processos zombie são evitados com *signal(SIGCHLD, SIG_IGN)*. A autenticação é feita por PSK: o cliente envia uma *RegisterMessage* com a sua chave; o servidor verifica no ficheiro *psk.txt* se o par (IP, chave) é válido e responde com mensagem de aceitação ou rejeição.

A função *close_protocol()* termina a stack de comunicação: notifica o servidor via TCP com a mensagem "SAIR", cancela a thread de multicast, sai do grupo multicast com *IP_DROP_MEMBERSHIP* e fecha todos os sockets abertos.

3.2 Difusão de Configuração via Multicast

Quando um cliente pretende alterar a configuração do protocolo, invoca *request_protocol_config()*, que envia uma *ConfigMessage* via TCP ao servidor com os novos parâmetros (retransmissão, backoff, sequência, timeout base e max_retries). O servidor recebe esta estrutura, interpreta os valores e retransmite-a integralmente via multicast para o grupo 239.0.0.1 na porta 9002, com TTL = 4 para garantir que o pacote atravessa os três routers Cisco do cenário. A configuração dos routers segue os comandos do enunciado: *ip multicast-routing* na configuração global e *ip pim sparse-dense-mode* em cada interface ativa.

Cada cliente tem uma thread dedicada em background que subscreve o grupo multicast com *IP_ADD_MEMBERSHIP* (conforme os slides TP06) e fica bloqueada em *recvfrom()* à espera de mensagens. Quando chega uma nova *ConfigMessage* do servidor, a thread atualiza imediatamente a configuração ativa do protocolo — sem necessidade de reiniciar ligações ou processos.

4. Testes e Resultados

4.1 Registo e Autenticação de Clientes

Os três clientes (Cliente1: 10.5.2.1, Cliente2: 10.5.2.2, Cliente3: 193.137.100.1) registaram-se com sucesso no servidor via TCP. O servidor verificou as PSKs no ficheiro *psk.txt* e confirmou cada registo com a mensagem de aceitação, registrando o IP e adicionando o cliente à lista de ativos.

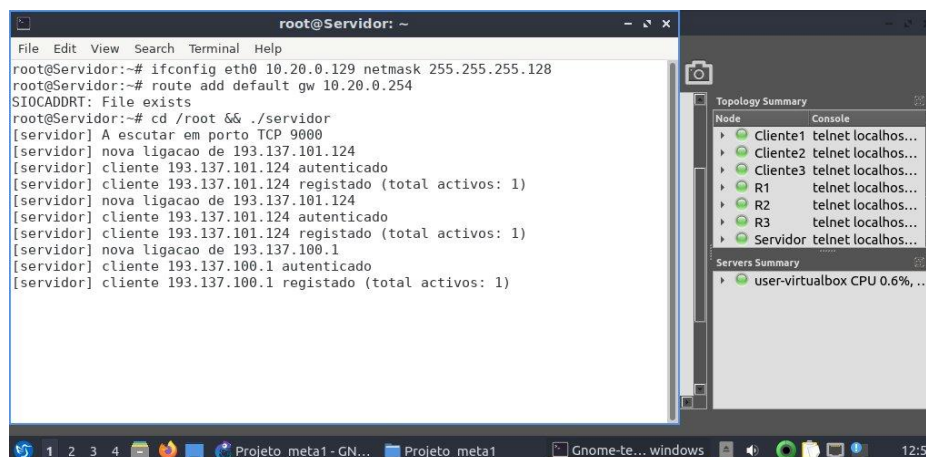
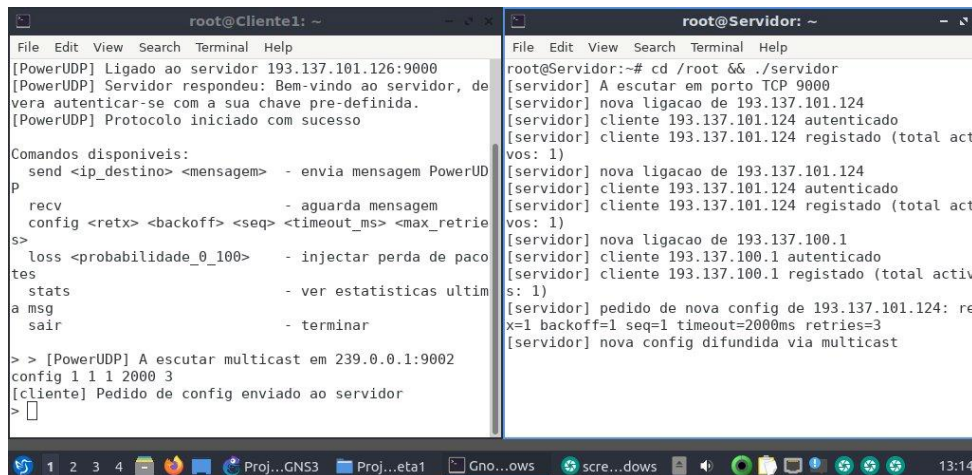


Figura 1 — Servidor com os 3 clientes autenticados e registados

4.2 Mudança Dinâmica de Configuração via Multicast

O Cliente1 enviou o pedido *config 1 1 2000 3* ao servidor, solicitando ativação de retransmissão, backoff e sequência, com timeout base de 2000 ms e máximo de 3 retransmissões. O servidor recebeu o pedido via TCP, construiu uma *ConfigMessage* e difundiu-a via multicast para o grupo 239.0.0.1:9002. As threads de multicast dos três clientes receberam a mensagem e atualizaram as respetivas configurações ativas em tempo

real, confirmado pela mensagem "*Nova config recebida via multicast*" em cada terminal. Este mecanismo permite alterar dinamicamente o comportamento do protocolo em toda a rede sem reiniciar qualquer processo.



```
root@Cliente1: ~
File Edit View Search Terminal Help
[PowerUDP] Ligado ao servidor 193.137.101.126:9000
[PowerUDP] Servidor respondeu: Bem-vindo ao servidor, de
vera autenticar-se com a sua chave pre-definida.
[PowerUDP] Protocolo iniciado com sucesso

Comandos disponiveis:
send <ip_destino> <mensagem> - envia mensagem PowerUD
p
rcv - aguarda mensagem
config <retx> <backoff> <seq> <timeout_ms> <max_retrie
s>
loss <probabilidade_0_100> - injectar perda de paco
tes
stats - ver estatisticas ultim
a msg
sair - terminar

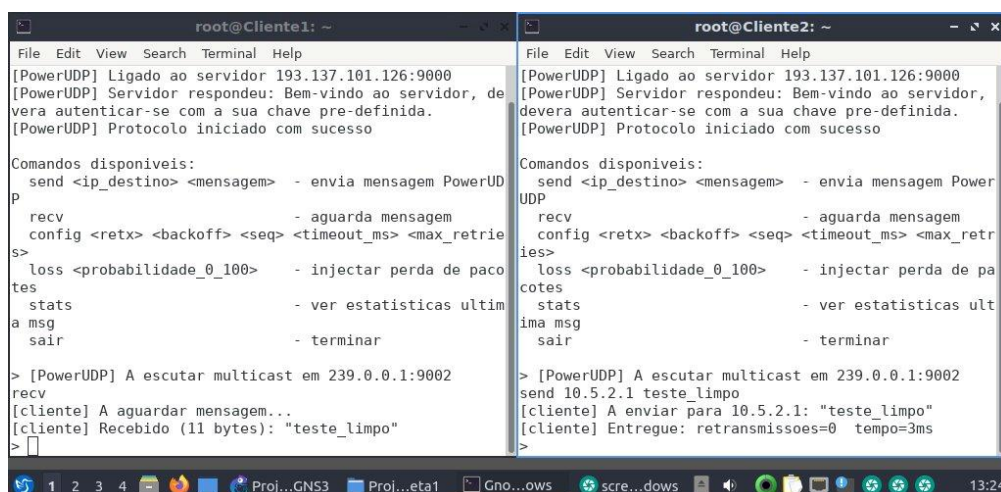
> > [PowerUDP] A escutar multicast em 239.0.0.1:9002
config 1 1 1 2000 3
[cliente] Pedido de config enviado ao servidor
>

root@Servidor: ~
File Edit View Search Terminal Help
root@Servidor:~# cd /root && ./servidor
[servidor] A escutar em porto TCP 9000
[servidor] nova ligacao de 193.137.101.124
[servidor] cliente 193.137.101.124 autenticado
[servidor] cliente 193.137.101.124 registado (total act:
vos: 1)
[servidor] nova ligacao de 193.137.101.124
[servidor] cliente 193.137.101.124 autenticado
[servidor] cliente 193.137.101.124 registado (total act:
vos: 1)
[servidor] nova ligacao de 193.137.100.1
[servidor] cliente 193.137.100.1 autenticado
[servidor] cliente 193.137.100.1 registado (total activi
s: 1)
[servidor] pedido de nova config de 193.137.101.124: re
x=1 backoff=1 seq=1 timeout=2000ms retries=3
[servidor] nova config difundida via multicast
```

Figura 2 — Cliente1 a enviar pedido de config; servidor a difundir via multicast

4.3 Comunicação entre Clientes com PowerUDP

O Cliente2 enviou uma mensagem ao Cliente1 sem perda de pacotes. A mensagem foi entregue com **0 retransmissões** em **3 ms**, demonstrando o funcionamento base do protocolo em condições normais de rede. O Cliente3 recebeu os dados corretos, enviou ACK e a confirmação foi recebida pelo emissor que incrementou o número de sequência.



```
root@Cliente1: ~
File Edit View Search Terminal Help
[PowerUDP] Ligado ao servidor 193.137.101.126:9000
[PowerUDP] Servidor respondeu: Bem-vindo ao servidor, de
vera autenticar-se com a sua chave pre-definida.
[PowerUDP] Protocolo iniciado com sucesso

Comandos disponiveis:
send <ip_destino> <mensagem> - envia mensagem PowerUD
p
rcv - aguarda mensagem
config <retx> <backoff> <seq> <timeout_ms> <max_retrie
s>
loss <probabilidade_0_100> - injectar perda de paco
tes
stats - ver estatisticas ultim
a msg
sair - terminar

> [PowerUDP] A escutar multicast em 239.0.0.1:9002
rcv
[cliente] A aguardar mensagem...
[cliente] Recebido (11 bytes): "teste_limpo"
>

root@Cliente2: ~
File Edit View Search Terminal Help
[PowerUDP] Ligado ao servidor 193.137.101.126:9000
[PowerUDP] Servidor respondeu: Bem-vindo ao servidor,
devera autenticar-se com a sua chave pre-definida.
[PowerUDP] Protocolo iniciado com sucesso

Comandos disponiveis:
send <ip_destino> <mensagem> - envia mensagem Power
UDP
rcv - aguarda mensagem
config <retx> <backoff> <seq> <timeout_ms> <max_retr
ies>
loss <probabilidade_0_100> - injectar perda de pa
cotes
stats - ver estatisticas ult
ima msg
sair - terminar

> [PowerUDP] A escutar multicast em 239.0.0.1:9002
send 10.5.2.1 teste_limpo
[cliente] A enviar para 10.5.2.1: "teste_limpo"
[cliente] Entregue: retransmissoes=0 tempo=3ms
>
```

Figura 3 — Envio Cliente2→Cliente1: 0 retransmissões, 3 ms

4.4 Injeção de Perda de Pacotes e Exponential Backoff

Com *inject_packet_loss(80)* ativo, o protocolo executou 5 retransmissões com timeouts crescentes (500 → 1000 → 2000 → 4000 → 8000 ms), totalizando **15 511 ms** até falha definitiva. Este resultado confirma o correto funcionamento da exponential backoff.

Após desativar a injeção de erros, a entrega voltou imediatamente a 3 ms com 0 retransmissões. A injeção de perda foi implementada com um gerador de números aleatórios: antes de cada *sendto()*, é gerado um valor entre 0 e 99 com *rand() % 100*. Se esse valor for inferior à probabilidade configurada, o pacote é descartado artificialmente sem ser enviado, simulando perda de rede.

Os valores de retransmissões e tempo de entrega apresentados nos testes são devolvidos pela função *get_last_message_stats(int *retransmissions, int *delivery_time)*, que acede ao estado interno do protocolo após cada envio.

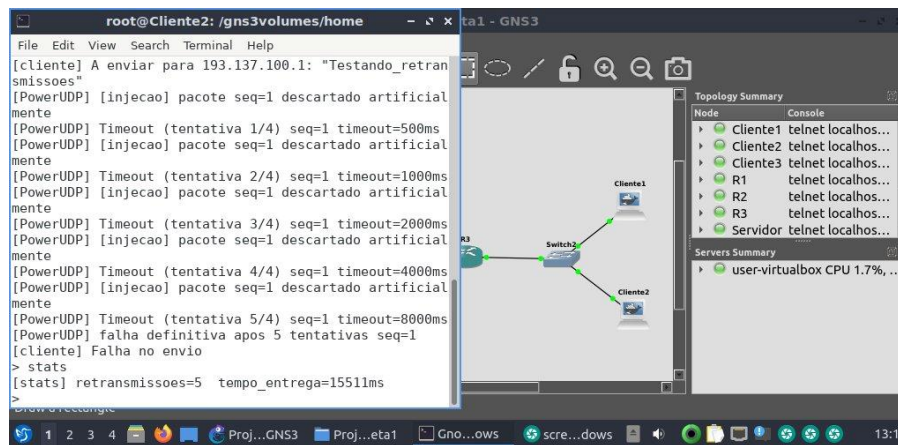


Figura 4 — Loss 80%: backoff 500→1000→2000→4000→8000 ms, stats: 5 retx, 15511 ms

Cenário	Retransmissões	Tempo entrega	Observação
Sem perda, retx ativa	0	3 ms	ACK recebido no 1.º envio
Loss 50%, retx ativa	1	~536 ms	1 retransmissão, timeout 500 ms
Loss 80%, retx ativa	5 (máx.)	15 511 ms	Backoff: 500+1000+2000+4000+8000 ms

4.5 Rejeição de Duplicados (NAK)

O recetor rejeitou corretamente mensagens com número de sequência repetido, enviando NAK ao emissor com indicação do número esperado vs. recebido ("*esperava seq=1, recebi seq=0*"). O emissor retransmitiu imediatamente sem incrementar o contador de tentativas falhadas, garantindo a integridade da ordenação.

5. Conclusão

O protocolo PowerUDP foi implementado com sucesso, cumprindo todos os requisitos do enunciado. A solução demonstrou ser funcional em ambiente real (GNS3 com routers Cisco e containers Docker), com comunicação entre clientes via UDP fiável, autenticação por PSK via TCP e reconfiguração dinâmica da rede via multicast.

Os testes realizados confirmaram o correto funcionamento de todos os mecanismos: o *exponential backoff* reagiu corretamente à perda de pacotes (500 → 1000 → 2000 → 4000 → 8000 ms), a ordenação por número de sequência rejeitou duplicados com NAK, e a mudança de configuração via multicast propagou-se a todos os clientes em tempo real sem reiniciar qualquer processo. A diferença de tempos de entrega entre condições normais (3 ms) e com 80% de perda (15 511 ms) ilustra claramente o impacto e a utilidade dos mecanismos implementados.